# EzExam — VIT Examination Portal

## End-to-End Project Report and Technical Handover

Prepared: 25 October 2026

**Live demo:** https://ez-exam-vit.vercel.app

**Repository:** github.com/NiharTambe20/EzExam-VIT

A clear technical handover explaining the project, architecture, workflows, fixes, deployment, testing, and remaining checkpoints.

# 1. Executive Summary

EzExam is a web-based examination portal built to support online assessments. Students can sign in, find available examinations, answer questions within a time limit, submit their attempts, and review results. Administrative users have separate functions for managing examinations and reviewing performance, based on the functionality implemented in the project.

The project uses a React frontend, a Java servlet-based backend, and a MySQL database. The frontend is hosted on Vercel, the backend is deployed on Render using Apache Tomcat in a Docker container, and the database is hosted on Aiven. The source code is maintained in GitHub.

A major part of the recent work was stabilizing exam timing and submission behavior. A timezone mismatch could make the countdown timer appear expired immediately, which could trigger an empty submission. The frontend timestamp parsing and timer safeguards were updated, tested, committed, and deployed. A later security audit also found authentication and IDOR risks in student endpoints; a remediation was reported as implemented with regression tests, but the report had not yet been independently reviewed at the time this handover was prepared.

# 2. Project Goals and User Experience

## What the project is for

EzExam aims to provide a single web application for conducting online examinations. It separates the user interface (what people see) from server-side rules (what the application is allowed to do) and persistent storage (where examination data is saved).

## Student workflow

- Open the deployed application and sign in.
- View available/published examinations and choose an exam.
- Read the exam instructions or pre-exam briefing, where provided.
- Answer questions and move between them using the question navigation controls.
- Use the countdown timer to track remaining time.
- Submit manually or allow the system's expiry behavior to submit the attempt.
- Review the scorecard and previous results, subject to authentication and permissions.

## Administrator workflow

The code and README update report describe an administrator console with exam lifecycle controls, question authoring, summary analytics, recent-attempt information, and student performance views. Exact capabilities should be understood from the current source code; this report records the features reported in the project rather than asserting that every workflow has been independently retested in production.

# 3. Technology Stack in Plain Language

| Layer | Technology | What it does |
|---|---|---|
| Frontend | React 19 | Builds the interactive pages and components users see. |
| Frontend tooling | Vite 8 | Runs the frontend development server and creates production assets. |
| Routing and icons | React Router DOM 7, Lucide React | Moves between pages and supplies interface icons. |
| Styling | CSS / Tailwind CSS v4 (as reported) | Controls layout, spacing, colors, and responsive appearance. |
| Backend | Java 17 | Runs the server-side application logic. |
| Web/API layer | Jakarta Servlets 6.0 | Receives HTTP requests and returns API responses. |
| Database access | JDBC and HikariCP | Connects Java code to MySQL; HikariCP reuses database connections efficiently. |
| Application server | Apache Tomcat 10.1 | Runs the servlet-based Java web application. |
| Build tool | Maven | Downloads Java dependencies, runs tests, and packages the backend. |
| Database | MySQL | Stores users, exams, questions, options, attempts, and submitted answers. |
| Frontend hosting | Vercel | Publishes the React application and provides the frontend deployment. |
| Backend hosting | Render + Docker | Runs the Java/Tomcat backend in a container. |
| Database hosting | Aiven | Provides the hosted MySQL database. |

# 4. High-Level Architecture

The application is split into three main runtime layers. The browser downloads the React application from Vercel. API requests are routed to the Java backend running on Render. The backend checks authentication and permissions, applies exam rules, reads or writes data through JDBC, and communicates with MySQL on Aiven.

```
Student or Administrator Browser
■
▼
React + Vite Frontend (Vercel)
■ /api/* requests
▼
Vercel API Proxy / Reverse Proxy
■
▼
Java 17 + Jakarta Servlets on Tomcat 10.1 (Render/Docker)
■ JDBC + HikariCP
▼
MySQL Database (Aiven)
```

The proxy keeps the frontend's API calls under the expected /api path and forwards them to the Render backend. Hosting the frontend, backend, and database separately means each has its own deployment and configuration.

# 5. Main Application Components

## Frontend

The frontend is a single-page application (SPA). React components render the login screen, student/admin pages, examination interface, timer, question palette, and result views. A service layer makes API calls. Browser session storage is used for in-progress answer persistence so a page interaction or refresh can retain locally stored answers, while final evaluation and saved results are handled by the backend.

## Backend

The backend is a Java web application built around Jakarta Servlets. Servlets receive requests and coordinate with service and DAO classes. Service classes hold workflows such as evaluating and submitting an exam. DAO (Data Access Object) classes isolate SQL queries and database operations from request handling. Models represent application data, and authentication/session utilities identify the signed-in user.

### Database

The schema reported in the repository contains six main tables: users, exams, questions, options, attempts, and attempt_answers. In simple terms, users represent accounts; exams represent assessments; questions and options define the content; attempts represent a student's sitting of an exam; and attempt_answers store answers submitted for that attempt. The schema and seed script support local setup and sample data.

### Connection pooling

HikariCP manages a pool of reusable database connections. Reusing connections is generally faster and more efficient than opening a new connection for every database operation.

## 6. Exam Lifecycle: From Start to Result

- Authentication: the user signs in, and the backend creates or maintains a server-side HTTP session.
- Exam discovery: the frontend requests available examinations from the backend.
- Start/resume: the backend identifies the authenticated student and creates or resumes the appropriate attempt.
- Question display: the frontend renders questions and answer choices received from the API.
- Answer selection: the UI tracks selected answers and uses browser session storage for local persistence where implemented.
- Countdown: the frontend parses the attempt start time and exam duration, then updates the displayed time remaining.
- Submission: the frontend sends the attempt's answers to the backend, either following manual submission or legitimate timer expiry.
- Evaluation: the backend evaluates answers server-side and calculates the result according to the implemented scoring rules, including negative marking where configured.
- Persistence: attempt status, answers, and result information are stored in MySQL.
- Result review: the frontend requests the saved result and presents the scorecard and attempt history permitted for the signed-in user.

## 7. Important Bug Fixed: Timer Timezone and Premature Submission

A production issue was observed where an exam could open with a timer showing 00:00 or 'Expiring Soon'. The result could then show every question as unanswered. These symptoms were connected: the timer could treat the attempt as already expired and trigger submission before the student had time to answer.

### Root cause

The backend produced a timezone-free SQL-style timestamp, such as a date and time without a UTC offset. The frontend parsed that value as local browser time. On an India Standard Time (IST) browser, a timestamp intended to represent UTC could therefore be interpreted as a different instant, making the start time appear several hours in the past.

### Frontend changes

- In `frontend/src/utils/timerUtils.js`, timezone-free server datetime strings are normalized as UTC by adding the `Z` suffix; explicit timezone offsets and numeric epoch timestamps are handled separately.
- In `frontend/src/components/Timer.jsx`, guards prevent missing or invalid start-time/duration props from triggering premature expiration. The expiration callback is guarded to run once for a genuinely expired attempt.
- In `frontend/test/examPersistenceAndTimer.test.js`, regression tests were added or updated to cover timestamp parsing and timer behavior.

### Validation and deployment

The reported results were 13 frontend tests passing, a successful Vite production build, and 32 backend Maven tests passing. The changes were committed with message `fix exam timer timezone and prevent premature submission` and commit `4bca080`, then the Vercel deployment was shown as Ready. A fresh exam attempt was subsequently tested by the user, who reported that it worked correctly.

This is an example of why a timer bug can become a data-integrity problem: an incorrect time calculation can cause an early submission, and an already-submitted attempt may not be safely replaceable by a later submission.

# 8. Performance and Reliability Work

Earlier work focused on database performance and reliable exam submission. The project added HikariCP connection pooling, optimized question/option retrieval, and improved submission/scoring transaction handling and idempotency. Idempotency means a repeated submission request should not accidentally score or store the same attempt multiple times.

The earlier combined backend/frontend work was committed as `a9dc7c3` with message `optimize database performance and fix exam submission reliability`. Subsequent deployment configuration work included Render/Docker setup and a Vercel API proxy configuration. Commit `1bf2f3a` was reported for Render deployment configuration, and `4e4bb4d` for a Vercel API proxy fix. The exact current branch history should be checked in GitHub if a future code change depends on these commits.

# 9. Security Audit and Reported Remediation

## What the audit found

The pre-publication audit identified an authentication bypass/IDOR risk in student exam and result endpoints. In the reported vulnerable implementation, if there was no authenticated session, some code paths could fall back to a `userId` supplied by a request body or query parameter. A caller could potentially pretend to be another student. Another path reportedly skipped attempt ownership checks when the user was unauthenticated.

## What the remediation report says was changed

- The three affected servlets were reported as updated: `StudentAttemptServlet.java`, `StudentExamServlet.java`, and `StudentResultServlet.java`. Student operations now require a session, use the session's authenticated user ID, and enforce attempt ownership.
- Conflicting client-supplied identities are reportedly rejected with HTTP 403; unauthenticated requests are rejected with HTTP 401.
- A new `AuthenticationSecurityTest.java` suite reportedly covers unauthenticated requests, forged IDs, cross-student access, student workflows, administrator workflows, and session expiration.

## Reported test results and important caveat

The remediation report stated that 42 backend tests and 13 frontend tests passed. However, its test output showed a MySQL connection, and it printed BCrypt test hashes. Before relying on the tests, verify that the database was an isolated test database and avoid publishing unnecessary credential-related output. The security changes were described as 'ready for review', not independently verified in the conversation. Therefore, this report does not claim that the vulnerability is conclusively fixed or that the security changes are already deployed.

Before the repository or backend is treated as secure, review the actual diff, inspect the test database configuration, and verify that the tests do not modify production data. Also confirm that the security fix has been committed, pushed, and deployed if the live backend is expected to contain it.

## Other defense-in-depth items

The audit also mentioned possible future improvements: explicit session timeout policy, CSRF protection for state-changing requests, and login rate limiting. These were recommendations, not confirmed completed features.

# 10. Repository and Documentation Preparation

The repository is hosted at GitHub. The README was reported as updated to explain the project, live demo, technology stack, architecture, setup, environment variables, API endpoints, testing, deployment, academic disclaimer, license, and contributors.

The README update report also said that local machine paths and broken `file:///Users/...` links were removed from the root README, and that the live deployment architecture replaced outdated text suggesting deployment had not happened.

The repository contains a `LICENSE` file reported to be MIT. The README report lists Nihar Tambe and Gaurang Khochare as contributors. The repository's actual visibility was not confirmed in the conversation; do not assume it is public until you check GitHub settings or open it in a logged-out browser.

A pre-publication audit also found demo credentials in seed/UI-related code and noted that local sample credentials must never be reused in production. It identified local filesystem paths in older internal reports and machine-specific paths in `setup-db.sh`. The user elected not to prioritize those non-critical cleanup items. The authentication vulnerability, however, was identified as critical and should be resolved before public release.

# 11. Deployment and Useful Links

| Resource | URL | Use |
|---|---|---|
| Live frontend | https://ez-exam-vit.vercel.app | Open the deployed student/admin interface. |
| Backend base URL | https://ezexam-backend.onrender.com | Base address for the deployed API service. |
| GitHub repository | https://github.com/NiharTambe20/EzExam-VIT | Source code, commits, issues, and clone/fork access subject to visibility. |
| Vercel dashboard | https://vercel.com/dashboard | Manage frontend deployments and logs. |
| Render dashboard | https://dashboard.render.com | Manage backend service and logs. |
| Aiven console | https://console.aiven.io | Manage the hosted MySQL service. |

The backend was reported as running on Render's Free tier at the time of deployment. Free services may sleep or take longer to respond on the first request. This is a hosting-tier behavior, not necessarily an application error.

Never store hosting dashboard credentials, database passwords, API keys, or deployment tokens in the public repository. The public URL is not itself a secret; the backend must be secured through authentication, authorization, and sensible operational controls.

# 12. Running the Project Locally

These are the general setup steps. Exact environment variable names and commands should be checked against the current README, `package.json`, Maven configuration, database configuration examples, Dockerfile, and setup script before following them verbatim.

## Prerequisites

- Git to clone the repository.
- Node.js and npm to run and build the frontend.
- Java 17 and Maven to build and test the backend.
- MySQL for local database storage, unless using a specifically configured alternative.
- Docker or Apache Tomcat 10.1 for running the Java web application, according to the chosen workflow.

## Clone the repository

```
git clone https://github.com/NiharTambe20/EzExam-VIT.git
cd EzExam-VIT
```

## Frontend

```
cd frontend
npm install
npm run dev
```

Use the scripts actually defined in `frontend/package.json`. The local Vite proxy configuration may route `/api/*` calls to the backend, so ensure the backend is running and the local proxy target is correct.

## Backend

```
cd backend
mvn test
mvn package
```

Configure the database connection using the documented local configuration template or environment variables. Do not copy real production credentials into source files. The packaging command produces the backend artifact described by the Maven project; deploy it to the configured Tomcat/Docker workflow.

## Database

Review `backend/src/main/resources/schema.sql` to create the database structure and `backend/src/main/resources/seed.sql` to load optional sample records. Use a dedicated local/test database. Never point test scripts at the production database. Demo credentials are for local evaluation only and must be changed or disabled for any real deployment.

# 13. Testing and Quality Checks

| Test/build check | Reported outcome | Notes |
|---|---|---|
| Frontend tests (`npm test`) | 13 passed | Reported after the timer fix and again in the security remediation report. |
| Frontend production build (`npm run build`) | Passed | Vite build succeeded after the timer fix. |
| Backend Maven tests (`mvn test`) | 32 passed before security remediation | Reported for the timer/submission release. |
| Backend Maven tests after security remediation | 42 passed, 0 failures/errors | Reported by Antigravity; independent review and database isolation confirmation were still pending. |
| Fresh exam attempt | User reported success | Timer/answer/scoring flow was tested after the frontend deployment. |

Test counts change as tests are added. When working on the project again, run the current test commands and use their actual output rather than relying only on historical counts.

# 14. Git and Release Timeline

| Milestone | Commit / reference | Summary |
|---|---|---|
| Render deployment setup | `1bf2f3a` (reported) | Backend deployment configuration and related packaging setup. |
| Vercel API proxy fix | `4e4bb4d` (reported) | Frontend API requests routed through the Vercel proxy. |
| Performance/submission reliability | `a9dc7c3` (reported) | Connection pooling, query improvements, and submission/scoring reliability work. |
| Timer timezone fix | `4bca080` | Fixed UTC timestamp interpretation and premature timer expiration; deployed to Vercel. |
| Security remediation | Not confirmed committed | Three student servlets and a new security test suite were reported changed; final review and push/deployment status were not confirmed. |
| README public-release update | Not confirmed committed | Root README was reported updated; no commit/push confirmation was provided. |

Treat commit hashes and status entries as a historical record from the conversation. Before continuing work, run `git status` and inspect the current GitHub `main` branch to determine what is actually committed and deployed.

# 15. Known Risks, Decisions, and Open Items

- Security fix review: inspect the servlet diff and new tests; confirm the tests use isolated test data and do not touch production MySQL.
- Security deployment: confirm whether the authentication fix was committed, pushed, and deployed. The reported implementation is not the same as verified production protection.
- Public visibility: repository visibility was not confirmed. Check GitHub before sharing it as a public repository.
- Demo credentials: ensure default/sample accounts are local-only and cannot be used to access a production admin account.
- CSRF and rate limiting: recommended future defense-in-depth improvements; not confirmed implemented.
- Session timeout: review the timeout against expected exam duration and ensure session expiration behavior is appropriate.
- Branding: the README disclaimer describes the project as independent and academic; confirm that use of institutional assets is appropriate for your intended publication.
- Setup portability: `setup-db.sh` was reported to contain machine-specific Homebrew/Tomcat paths. Check the script before using it on another machine.
- Documentation accuracy: keep README endpoints, environment variables, test counts, screenshots, and setup commands synchronized with the actual code.

# 16. How to Resume This Project Later

If you return to EzExam after several days, use this checklist to regain context quickly.

- Open the local repository at `/Users/nihartambe/Desktop/ezexam`.
- Run `git status` to see uncommitted changes and `git log --oneline -10` to see recent commits.

- Check GitHub's `main` branch and the Vercel/Render deployment dashboards to confirm which code is live.
- Open the root README first for user-facing project documentation and setup instructions.
- For timer behavior, start with `frontend/src/utils/timerUtils.js`, `frontend/src/components/Timer.jsx`, and `frontend/test/examPersistenceAndTimer.test.js`.
- For authentication, start with the three student servlets and `AuthenticationSecurityTest.java`; verify the test database before rerunning tests.
- Run the frontend tests/build and backend Maven tests after checking the environment and database configuration.
- Never commit secrets or test against the production database.

The simplest mental model is: React displays the portal; Java servlets enforce rules and expose APIs; MySQL stores exam data; Vercel, Render, and Aiven host the three layers.

## 17. Glossary

| Term | Plain-language meaning |
|---|---|
| API | A defined way for the frontend and backend to exchange requests and responses. |
| SPA | Single-page application: the browser loads the app shell and React updates views without a full page reload for every navigation. |
| Servlet | A Java component that handles HTTP requests on the backend. |
| JDBC | Java's standard API for connecting to relational databases and executing SQL. |
| DAO | Data Access Object: code that handles database queries for a particular data area. |
| ORM | Object-relational mapping: a way to represent database records as application objects; do not assume a full ORM is used unless the code confirms it. |
| HTTP session | Server-side state associated with a browser session, commonly identified by a cookie. |
| IDOR | Insecure Direct Object Reference: a vulnerability where changing an object ID can expose or modify someone else's data because ownership is not checked. |
| CSRF | Cross-site request forgery: an attack that attempts to make a user's browser submit an unwanted request to a site where they are signed in. |
| Connection pool | A set of reusable database connections that avoids repeatedly creating new connections. |
| Vite build | The optimized set of frontend files created for deployment. |
| Docker image/container | A packaged environment used to build and run the backend consistently. |
| Deployment | Publishing and running the application on hosting infrastructure. |
| Regression test | A test that checks a previously working behavior remains correct after a code change. |
| Idempotency | A property that prevents repeating the same operation from causing unintended duplicate effects. |

## 18. Final Summary

EzExam is a three-layer web application: React/Vite for the interface, Java/Jakarta Servlets on Tomcat for backend logic, and MySQL for persistent data. It is deployed using Vercel, Render, and Aiven, with the source maintained in GitHub.

The team addressed deployment, database performance, submission reliability, and a production timer bug. The timer issue was linked to timezone-free timestamps being parsed as local time, and the fix was deployed and then tested with a fresh exam attempt.

The most important unresolved release checkpoint in the recorded history is the authentication/IDOR remediation: it was reported as implemented and covered by tests, but the independent code review, test database isolation confirmation, commit/push, and production deployment were not confirmed. The repository's public visibility was also not confirmed. This report deliberately preserves those distinctions so future work can resume from an accurate state.